

FedSparsy:

Reference Paper: SparsyFed: Sparse Adaptive Federated Training

<https://arxiv.org/abs/2504.05153>

1. Project Overview

FedSparsy is a federated learning framework designed to reduce both computation and communication overhead. It re-implements the core ideas of **SparsyFed** (Sparse Adaptive Federated Training), including dynamic activation pruning during local training, a sparse update mechanism for client-to-server communication, and per-layer weight re-parameterization.

The framework is built on top of an MPI-based distributed FL backbone and supports both CV (image classification) and NLP (language modeling) tasks.

2. Key Features

-Dynamic Activation Pruning: During the backward pass, layer-wise Top-K pruning is applied to activation gradients based on the reference model's weight density. This reduces backward computation proportionally to the model's sparsity.

-Sparse Model Updates: Clients compute $\Delta\omega = \omega_{\text{current}} - \omega_{\text{initial}}$ after local training and upload only the top-K% most significant entries (controlled by `--target_sparsity`), significantly reducing communication volume.

-Weight Re-parameterization: Applies $\theta = \text{sign}(\omega) \cdot |\omega|^\beta$ (controlled by `--reparam_beta`) before local optimization to improve gradient signal strength in sparse regions.

-Per-step Re-parameterization (`--reparam_per_step`): Re-parameterization is applied as temporary forward-time weights at every batch, rather than once per round. This is more aggressive and can accelerate convergence.

-Dense Start Strategy: The first round is always kept dense (no Top-K) to warm-start the global model before entering the sparse regime.

-Server-side Optimizer: Supports SGD/Adam on the server to apply aggregated sparse updates, with optional momentum and global norm clipping (`--server_clip_norm`).

3. Algorithm Overview

The training loop follows the SparsyFed paper:

```
1
2 For each round t:
3
4     1. server broadcasts global model  $\omega_t$  to selected clients
5
6     2. Each client:
```

```

7
8     a. [Optional] Re-parameterize weights:  $\theta = \text{sign}(\omega) * |\omega|^\beta$ 
9
10    b. Run local SGD for E epochs
11
12        - Backward gradients of activations are sparsified via Top-K per
layer
13
14    c. Compute update:  $\Delta\omega = \omega_{\text{local}} - \omega_t$ 
15
16    d. Apply global Top-K pruning to  $\Delta\omega$  with target sparsity s
17
18    e. Upload sparse  $\Delta\omega$  to server
19
20  3. Server:
21
22    a. Aggregate sparse updates:  $\Delta\omega_{\text{global}} = \sum (n_i / N) * \Delta\omega_i$ 
23
24    b. Apply server optimizer:  $\omega_{\{t+1\}} = \omega_t + \eta_s * \Delta\omega_{\text{global}}$ 
25
26    c. Evaluate on validation set every K rounds
27

```

4. Experimental Results

Setup

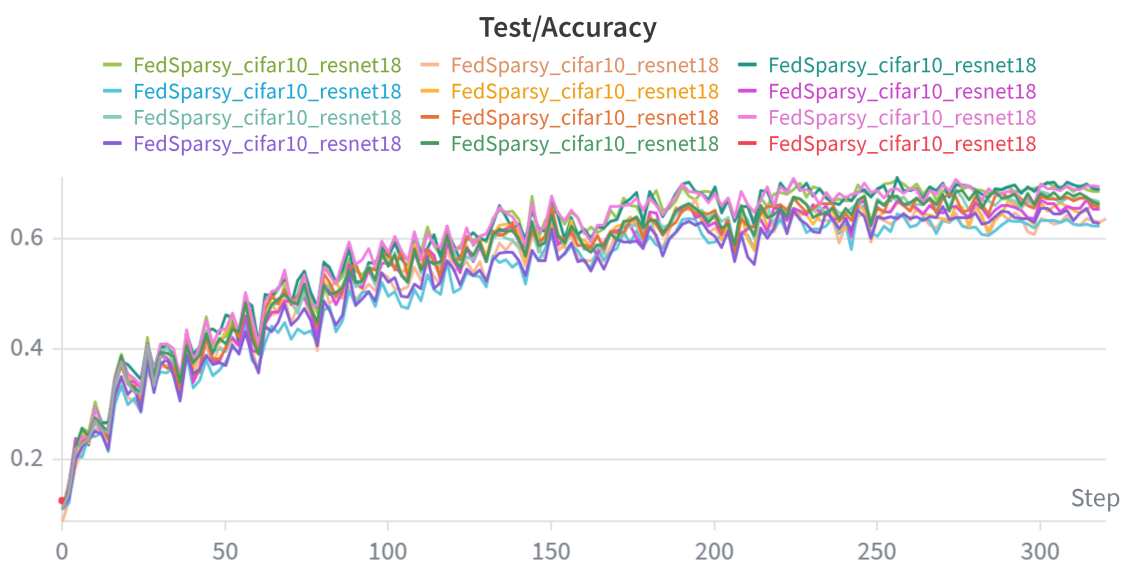
Config Value
----- -----
Model ResNet-18
Dataset CIFAR-10
Total clients 100
Clients per round 10
Communication rounds 500
Local epochs 5
Target density (weight) 0.5
Client LR 0.02
Target sparsity (upload) 0.8
Re-param beta (β) 1.1
Re-param per step $\checkmark$
Server LR 0.8
Server clip norm disabled (-1)
Batch size 64
Client optimizer SGD

| GPUs | 4× (CUDA 4,5,6,7) |

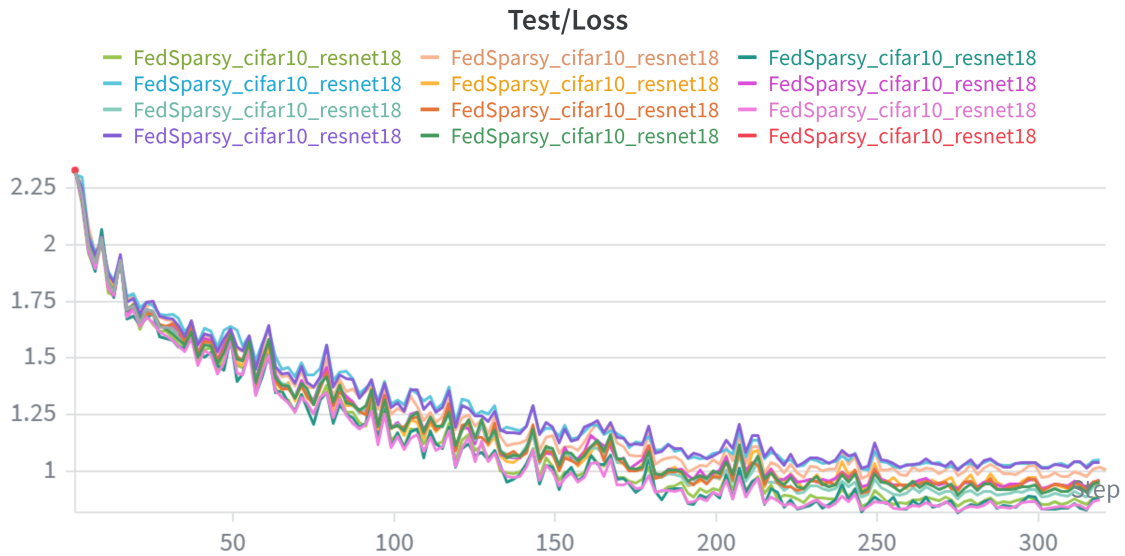
Launch command:

```
1
2 CUDA_VISIBLE_DEVICES=4,5,6,7bashrun_fedsparsy_distributed_pytorch.sh\
3
4 resnet18 cifar10 100 10 500 5 0.5 0.02 \
5
6 --gpu_mapping_keymapping_default\
7
8 --target_sparsity 0.8 \
9
10 --reparam_beta1.05\
11
12 --reparam_per_step \
13
14 --server_lr0.8\
15
16 --server_clip_norm -1 \
17
18 --num_eval512\
19
20 --frequency_of_the_test 5 \
21
22 --batch_size64\
23
24 --client_optimizer sgd
25
```

Convergence Observations



Test/Loss (W&B curve):



- Loss starts at ~2.3 (random initialization) and drops sharply in the first 10–20 rounds.
- Steady decrease continues through round ~150, converging to the range of **0.9–1.1** by round 300+.
- Curves from multiple runs are tightly clustered, indicating stable and reproducible training.

Test/Accuracy (W&B curve):

- Accuracy rises from ~0.1 to ~0.4 within the first 60 rounds.
- Continues climbing through round ~150, stabilizing at **0.62–0.70** for the final ~100 rounds.
- Most runs converge above **0.65**, with the best runs reaching ~**0.70**.
- The tight cluster of curves confirms low run-to-run variance.

Summary: Under 80% upload sparsity (only top-20% update entries are transmitted), FedSparsity achieves >70% test accuracy on CIFAR-10 with ResNet-18 in a 100-client heterogeneous federated setting, demonstrating effective sparse communication without significant accuracy degradation.

5. Problems Encountered & Solutions

Issue 1: Extremely Slow Loss Convergence

-Symptom: Loss stayed high or decreased very slowly.

-Root Cause: `--server_clip_norm` was set too low (e.g., 10), causing the server to scale down aggregated updates by ~95% (clip coefficient ~0.05).

-Solution: Set `--server_clip_norm -1` (disabled) or a much larger value to allow sufficient gradient flow in early rounds.

Issue 2: Process Hang on Termination (W&B Sync Issue)

-Symptom: Experiment finished locally but processes wouldn't exit, preventing W&B from syncing.

-Root Cause: MPI processes blocked in a message loop. The server did not explicitly signal clients to shut down.

-Solution:

- Implemented `MSG_TYPE_S2C_FINISH` signal broadcasted by the server.

- Added graceful exit: `Broadcast Finish → 0.5s Delay → Stop Comm Thread → MPI_Finalize`.
- Wrapped main loop in `try/finally` to ensure `wandb.finish()` is always called.

Issue 3: Training Instability in Early Rounds

-Symptom: High loss variance ("sawtooth" patterns) when starting with high sparsity from round 0.

-Solution: Implemented a **Dense Start** mechanism (round 0 always uses dense activations and full updates), stabilizing the global model before entering the sparse regime.

Issue 4: NaN/Inf Loss from Repeated Re-parameterization

-Symptom: Loss became NaN after a few batches.

-Root Cause: Re-parameterization ($\theta = \text{sign}(\omega) |\omega|^\beta$) was being applied cumulatively to the same parameters across batches, causing exponential blow-up.

-Solution: Switched to `functional_call` with a temporary copy of parameters — the original weights are never modified in-place, and re-parameterization is applied freshly at each forward pass.

Issue 5: MPI Runtime Errors (Double Free / Abort)

-Symptom: Occasional "Signal 6" or "Double Free" errors during shutdown.

-Root Cause: Race condition between manual `MPI_Finalize` and Python's garbage collection / `atexit` hooks.

-Solution: Explicitly stop background communication threads (`com_manager.stop_receive_message()`) before finalizing MPI to ensure clean resource release.

6. Hyperparameter Sensitivity Notes

Parameter	Effect	Recommendation
<code>--server_clip_norm</code>	Large / disabled → faster early convergence; too small → stagnation	Use <code>-1</code> unless gradient explosion observed
<code>--server_lr</code>	Controls how aggressively server applies aggregated updates	<code>0.8-1.0</code> works well for SGD clients
<code>--reparam_beta</code>	>1 amplifies large weights, sharpens sparse structure	<code>1.05-1.3</code> ; too large causes instability
<code>--reparam_per_step</code>	Per-batch re-param is more aggressive than per-round	Enable for faster convergence; disable if loss spikes
<code>--target_sparsity</code>	Higher → less communication, potentially lower accuracy	<code>0.8-0.9</code> is a reasonable trade-off
<code>--target_density</code>	Controls reference weight structure density	<code>0.5</code> means 50% weights kept in sparse layers

7. File Structure

```
1
2 api/distributed/fedsparsy/
3
4 |— FedSparsyAPI.py           # Entry point: init_server / init_client
5
6 |— FedSparsyAggregator.py    # Server-side aggregation + server
  optimizer
7
8 |— FedSparsyClientManager.py # Client MPI message handling
9
10 |— FedSparsyServerManager.py # Server MPI message handling
11
12 |— FedSparsyTrainer.py       # Client trainer: compute  $\Delta\omega$  + Top-K
  pruning
13
14 |— my_model_trainer_classification.py # Local training loop (CV tasks)
15
16 |— my_model_trainer_language_model.py # Local training loop (NLP tasks)
17
18 |— message_define.py         # MPI message type constants
19
20 |— utils.py                  # Tensor list/tensor conversion utilities
21
22
23 experiments/fedsparsy/
24
25 |— main_fedsparsy.py         # Argument parsing + FL pipeline entry
26
27 |— run_fedsparsy_distributed_pytorch.sh # Launch script with auto
  experiment logging
28
29 |— gpu_mapping.yaml          # GPU-to-process mapping
30
31 |— README.MD                 # Usage and parameter reference
32
```